Lab 5
Combination Circuits

Digital System Fundamentals

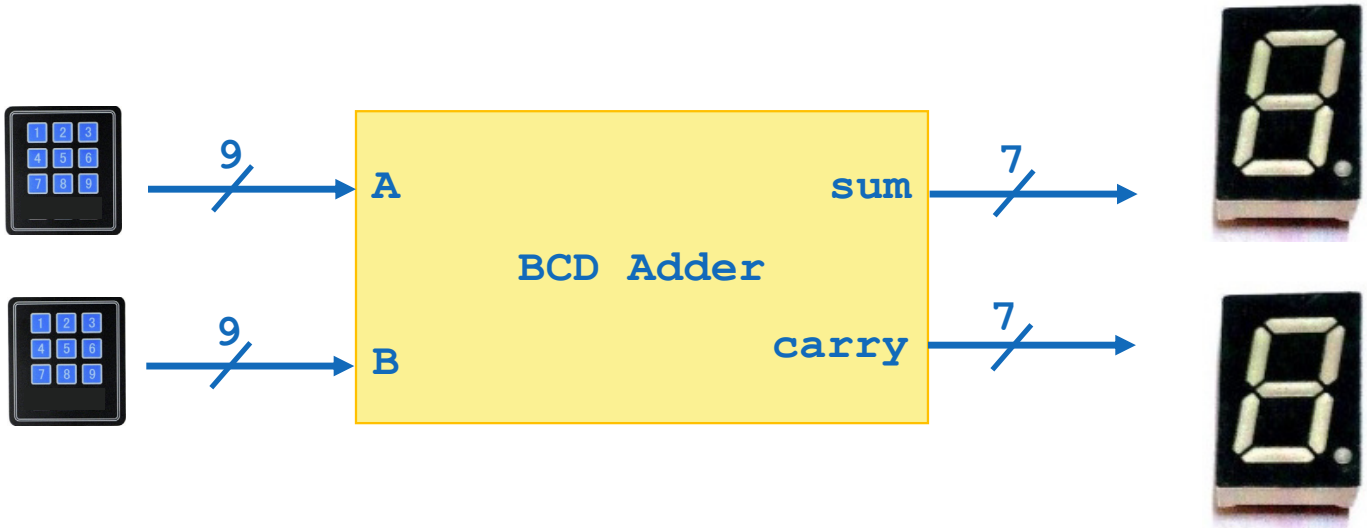
Sorayut Glomglome

Learning Outcomes

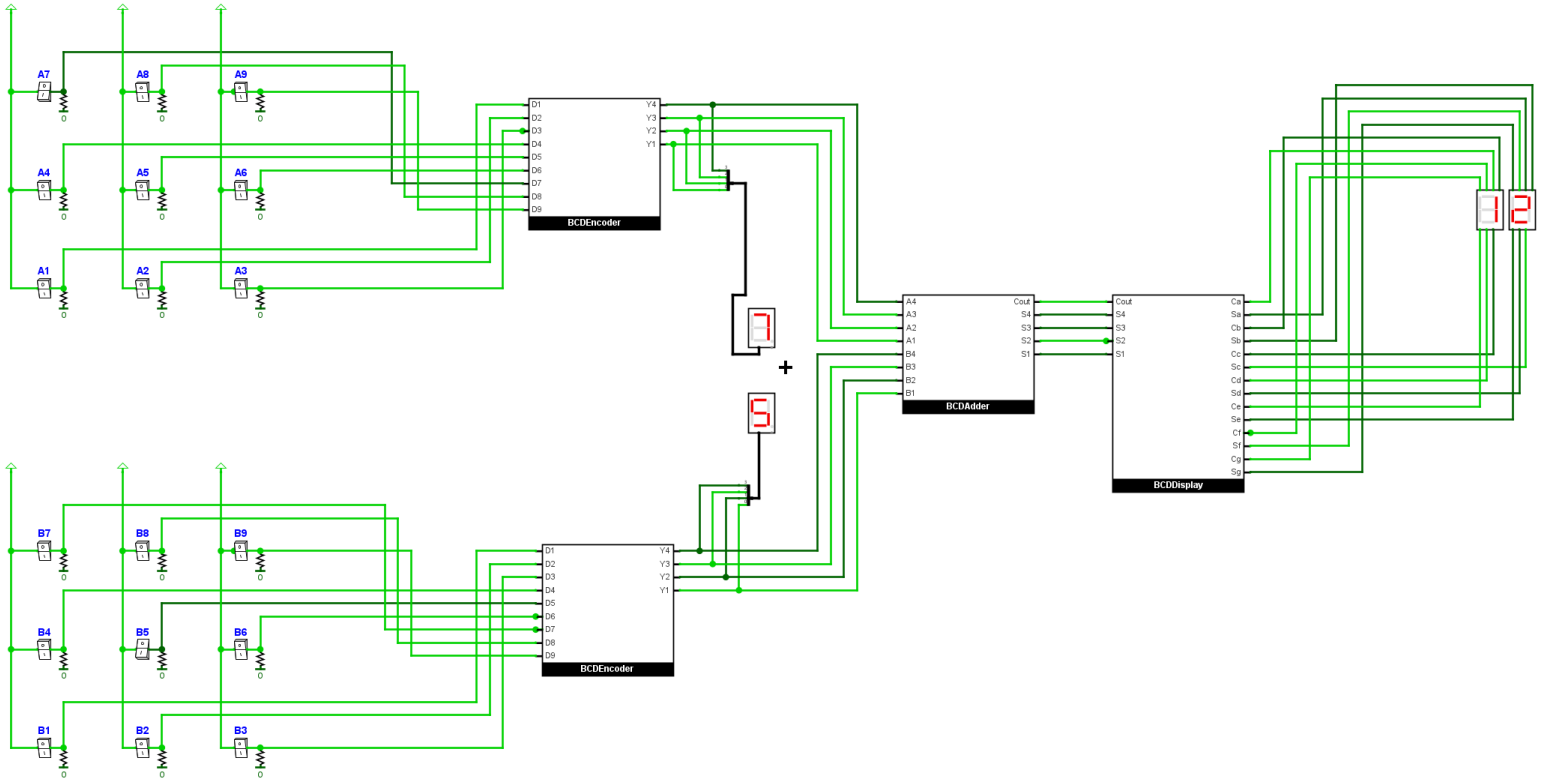
1. Use encoder/decoder to implement BCD Adder.
2. Implement basic ALU.

1. BCD Adder

BCD Adder | Top Level

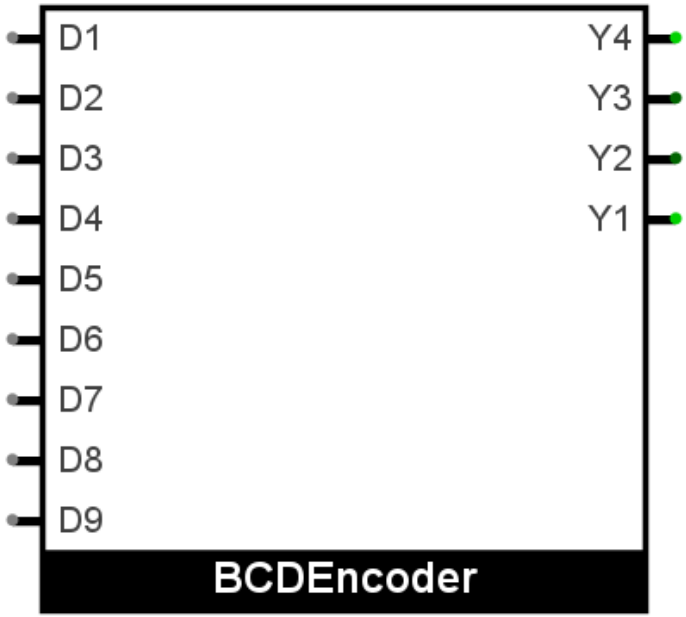


BCD Adder | Second Level



1.1 BCDEncoder Module

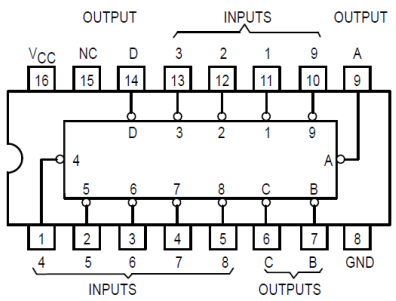
1. Create .csv file for the modified 74147-function table.
2. Build the subcircuit in Logisim by import truth table from .csv file.



1.1 BCDEncoder Module | 74147

74147 | 10-LINE-TO-4-LINE PRIORITY ENCODERS

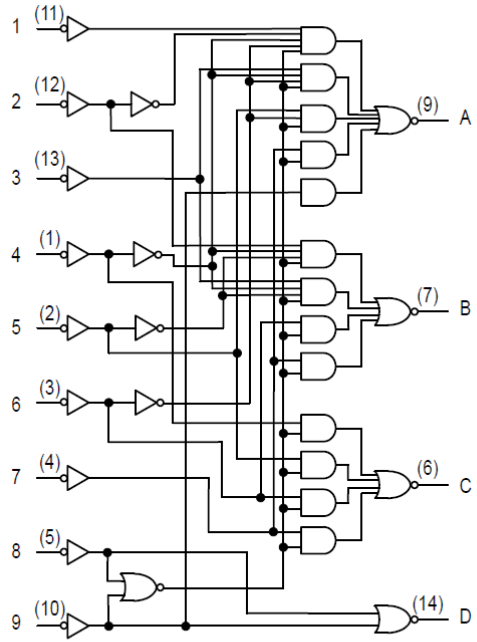
SN54/74LS147
(TOP VIEW)



SN54/74LS147
FUNCTION TABLE

INPUTS									OUTPUTS			
1	2	3	4	5	6	7	8	9	D	C	B	A
H	H	H	H	H	H	H	H	H	H	H	H	H
X	X	X	X	X	X	X	X	L	L	H	H	L
X	X	X	X	X	X	X	L	H	L	H	H	H
X	X	X	X	X	X	L	H	H	H	L	L	L
X	X	X	X	X	L	H	H	H	H	L	L	H
X	X	X	X	L	H	H	H	H	H	L	H	L
X	X	X	L	H	H	H	H	H	H	L	H	H
X	X	L	H	H	H	H	H	H	H	H	L	L
X	L	H	H	H	H	H	H	H	H	H	L	H
L	H	H	H	H	H	H	H	H	H	H	H	L

H = HIGH Logic Level, L = LOW Logic Level, X = Irrelevant



SN54/74LS147

1.1 BCDEncoder Module | .csv File

SN54/74LS147
FUNCTION TABLE

INPUTS									OUTPUTS			
1	2	3	4	5	6	7	8	9	D	C	B	A
H	H	H	H	H	H	H	H	H	H	H	H	H
X	X	X	X	X	X	X	X	L	L	H	H	L
X	X	X	X	X	X	X	L	H	L	H	H	H
X	X	X	X	X	X	L	H	H	H	L	L	L
X	X	X	X	X	L	H	H	H	H	L	L	H
X	X	X	X	L	H	H	H	H	H	L	H	L
X	X	X	L	H	H	H	H	H	H	L	H	H
X	X	L	H	H	H	H	H	H	H	H	L	L
X	L	H	H	H	H	H	H	H	H	H	L	H
L	H	H	H	H	H	H	H	H	H	H	H	L

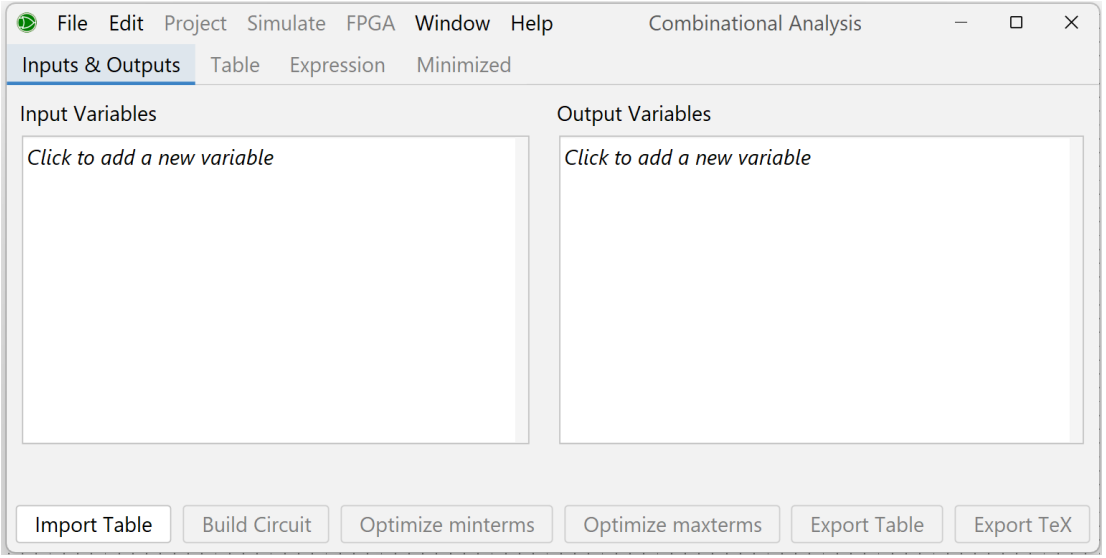
H = HIGH Logic Level, L = LOW Logic Level, X = Irrelevant

```
D1,D2,D3,D4,D5,D6,D7,D8,D9,|,Y4,Y3,Y2,Y1
1,1,1,1,1,1,1,1,1,|,0,0,0,0
-,-,-,-,-,-,-,-,0,|,1,0,0,1
-,-,-,-,-,-,-,0,1,|,1,0,0,0
-,-,-,-,-,-,0,1,1,|,0,1,1,1
-,-,-,-,-,0,1,1,1,|,0,1,1,0
-,-,-,-,0,1,1,1,1,|,0,1,0,1
-,-,-,0,1,1,1,1,1,|,0,1,0,0
-,-,0,1,1,1,1,1,1,|,0,0,1,1
-,0,1,1,1,1,1,1,1,|,0,0,1,0
0,1,1,1,1,1,1,1,1,|,0,0,0,1
```

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	D1	D2	D3	D4	D5	D6	D7	D8	D9	|	Y4	Y3	Y2	Y1
2	1	1	1	1	1	1	1	1	1	|	0	0	0	0
3	-	-	-	-	-	-	-	-	0	|	1	0	0	1
4	-	-	-	-	-	-	-	0	1	|	1	0	0	0
5	-	-	-	-	-	-	0	1	1	|	0	1	1	1
6	-	-	-	-	-	0	1	1	1	|	0	1	1	0
7	-	-	-	-	0	1	1	1	1	|	0	1	0	1
8	-	-	-	0	1	1	1	1	1	|	0	1	0	0
9	-	-	0	1	1	1	1	1	1	|	0	0	1	1
10	-	0	1	1	1	1	1	1	1	|	0	0	1	0
11	0	1	1	1	1	1	1	1	1	|	0	0	0	1

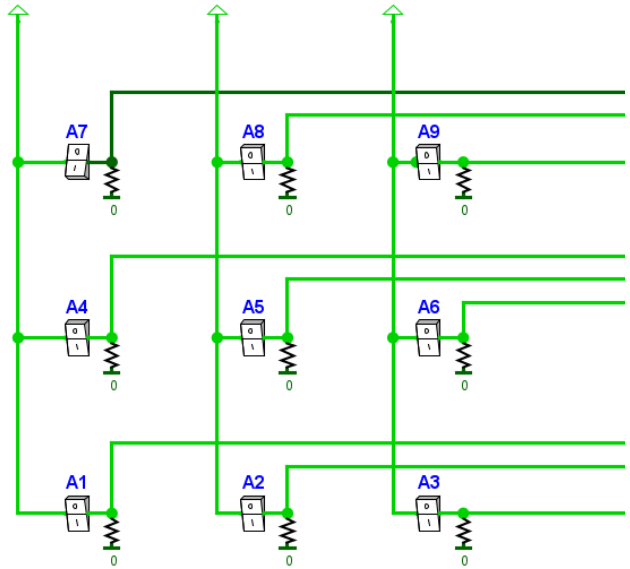
1.1 BCDEncoder Module | Implementation

- Menu Project -> Analyze Circuit.
- Import Table
- Select the created .csv file
- Optimize minterms
- Build Circuit
- Verify Circuit



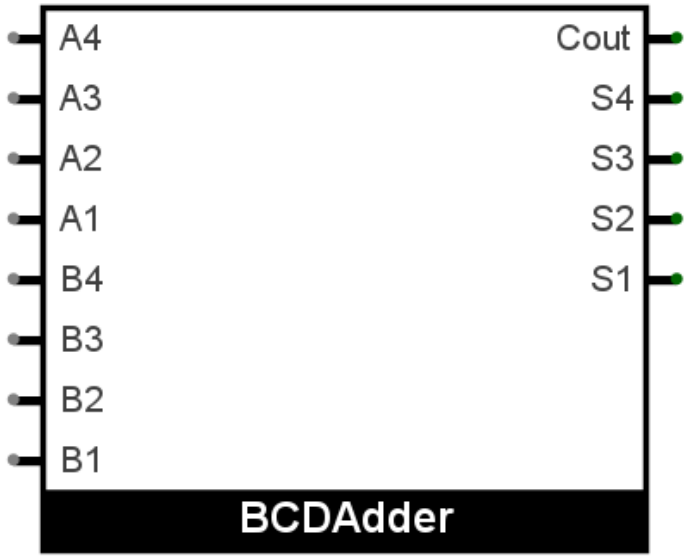
1.1 BCDEncoder Module | Keypad

- **Switch** component under **Input/Output Extra**.
- **Power** component under **Wiring**.
- **Pull Resistor** component under **Wiring**.
  - Set **Pull Direction** attribute to **Zero**.



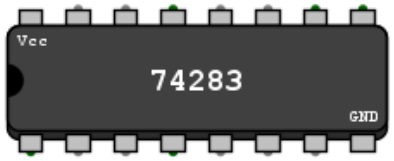
1.2 BCDAdder Module

1. Use 2 binary adder ICs.
2. Detect and adjust the summation.

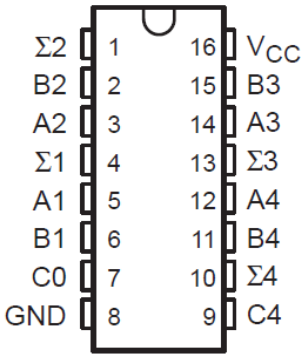


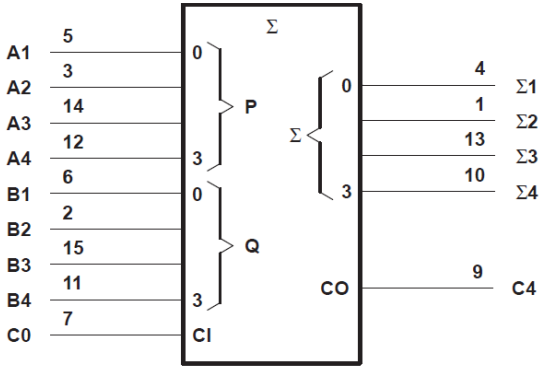
1.2 BCDAdder Module | 74283 Datasheet

- Use IC **74283** under **TTL**.



SN54F283 . . . J PACKAGE
SN74F283 . . . D OR N PACKAGE
(TOP VIEW)



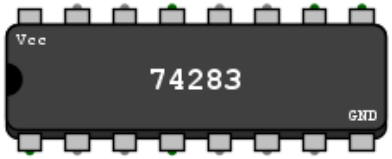


FUNCTION TABLE

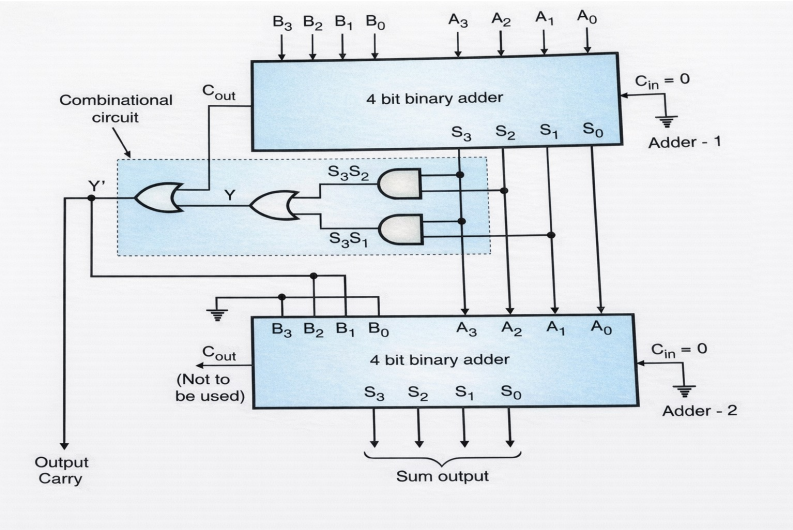
INPUTS				OUTPUTS					
				WHEN C0 = L			WHEN C0 = H		
				WHEN C2 = L			WHEN C2 = H		
A1	B1	A2	B2	Σ1	Σ2	C2	Σ1	Σ2	C2
A3	B3	A4	B4	Σ3	Σ4	C4	Σ3	Σ4	C4
L	L	L	L	L	L	L	H	L	L
H	L	L	L	H	L	L	L	H	L
L	H	L	L	H	L	L	L	H	L
H	H	L	L	L	H	L	H	H	L
L	L	H	L	L	H	L	H	H	L
H	L	H	L	H	H	L	L	L	H
L	H	H	L	H	H	L	L	L	H
H	H	H	L	L	L	H	H	L	H
L	L	L	H	L	H	L	H	H	L
H	L	L	H	H	H	L	L	L	H
L	H	L	H	H	H	L	L	L	H
H	H	L	H	L	L	H	H	L	H
L	L	H	H	L	L	H	H	L	H
H	L	H	H	H	L	H	L	H	H
L	H	H	H	H	L	H	L	H	H
H	H	H	H	L	H	H	H	H	H

NOTE: Input conditions at A1, B1, A2, B2, and C0 are used to determine outputs Σ1 and Σ2 and the value of the internal carry C2. The values at C2, A3, B3, A4, and B4 are then used to determine outputs Σ3, Σ4, and C4.

1.2 BCDAdder Module | Build & Adjust



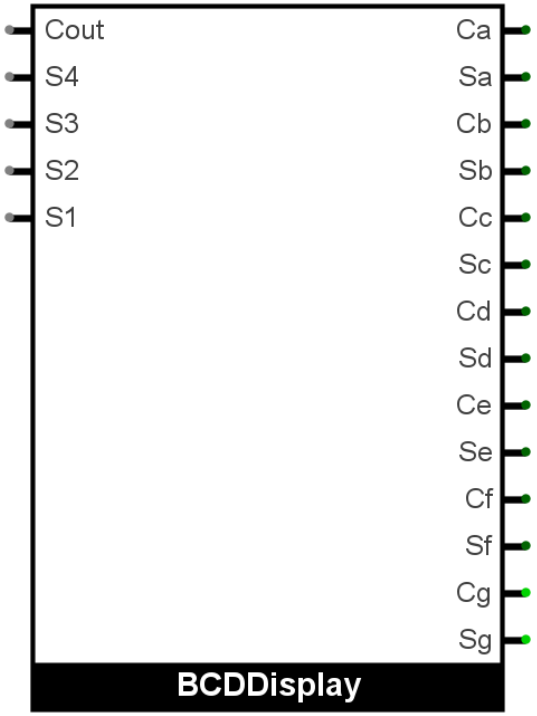
Case	Detail	Determine Operand $B_3\ B_2\ B_1\ B_0$	Action
1	Sum <= 9 & Carry = 0	0 0 0 0	No Adjusting
2	Sum > 9 & Carry = 0	0 1 1 0	+6
3	Sum <= 9 & Carry = 1	0 1 1 0	+6



S_3	S_2	S_1	S_0	Decimal	BCD	S_3S_2	S_3S_1	$S_3S_2 + S_3S_1$
0	0	0	0	0	0	00	00	0
0	0	0	1	1	1	00	00	0
0	0	1	0	2	2	00	01	0
0	0	1	1	3	3	00	01	0
0	1	0	0	4	4	01	00	0
0	1	0	1	5	5	01	00	0
0	1	1	0	6	6	01	01	0
0	1	1	1	7	7	01	01	0
1	0	0	0	8	8	10	10	0
1	0	0	1	9	9	10	10	0
1	0	1	0	10	-	10	11	1
1	0	1	1	11	-	10	11	1
1	1	0	0	12	-	11	10	1
1	1	0	1	13	-	11	10	1
1	1	1	0	14	-	11	11	1
1	1	1	1	15	-	11	11	1

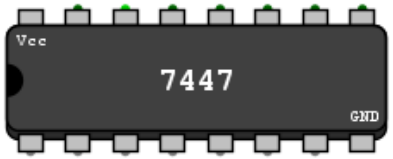
1.3 BCDDisplay

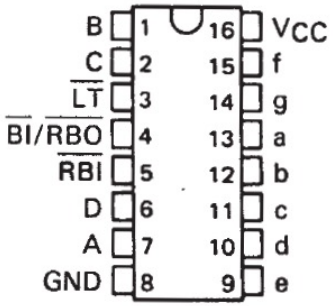
- Use 2 BCD-to-7SEG ICs.



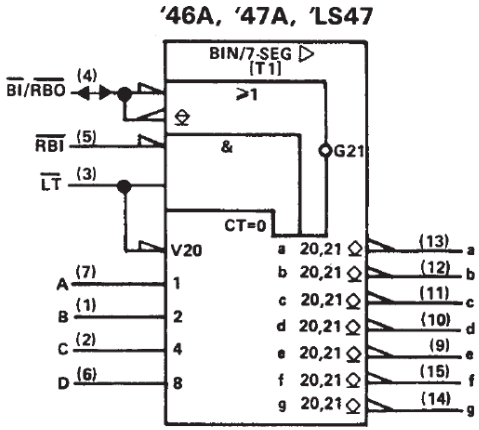
1.3 BCDDisplay Module | 7447 Datasheet

- Use IC **7447** under **TTL**.





logic symbols†



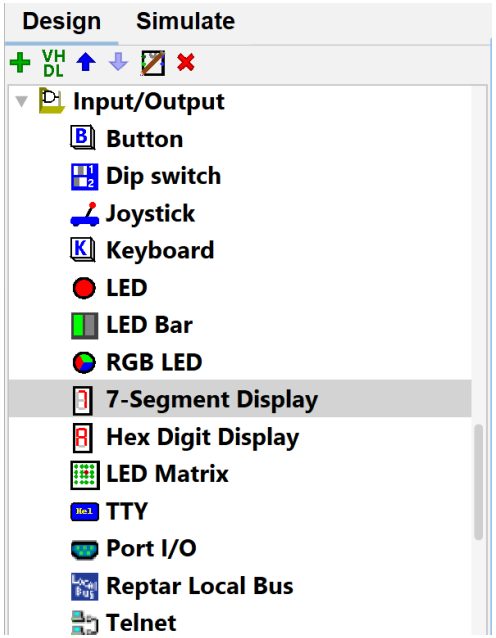
SEGMENT IDENTIFICATION

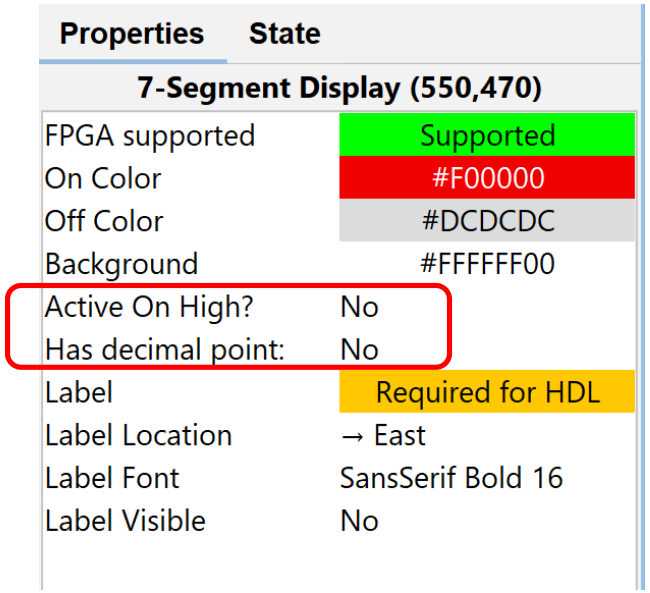
NUMERICAL DESIGNATIONS AND RESULTANT DISPLAYS

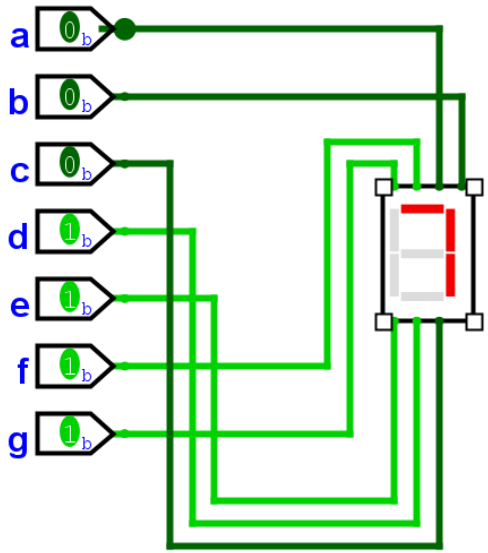
'46A, '47A, 'LS47 FUNCTION TABLE (T1)

DECIMAL OR FUNCTION	INPUTS						BI/RBO†	OUTPUTS							NOTE
	LT	RBI	D	C	B	A		a	b	c	d	e	f	g	
0	H	H	L	L	L	L	H	ON	ON	ON	ON	ON	ON	OFF	
1	H	X	L	L	L	H	H	OFF	ON	ON	OFF	OFF	OFF	OFF	
2	H	X	L	L	H	L	H	ON	ON	OFF	ON	ON	OFF	ON	
3	H	X	L	L	H	H	H	ON	ON	ON	ON	OFF	OFF	ON	
4	H	X	L	H	L	L	H	OFF	ON	ON	OFF	OFF	ON	ON	
5	H	X	L	H	L	H	H	ON	OFF	ON	ON	OFF	ON	ON	
6	H	X	L	H	H	L	H	OFF	OFF	ON	ON	ON	ON	ON	
7	H	X	L	H	H	H	H	ON	ON	ON	OFF	OFF	OFF	OFF	1
8	H	X	H	L	L	L	H	ON	ON	ON	ON	ON	ON	ON	
9	H	X	H	L	L	H	H	ON	ON	ON	OFF	OFF	ON	ON	
10	H	X	H	L	H	L	H	OFF	OFF	OFF	ON	ON	OFF	ON	
11	H	X	H	L	H	H	H	OFF	OFF	ON	ON	OFF	OFF	ON	
12	H	X	H	H	L	L	H	OFF	ON	OFF	OFF	OFF	ON	ON	
13	H	X	H	H	L	H	H	ON	OFF	OFF	ON	OFF	ON	ON	
14	H	X	H	H	H	L	H	OFF	OFF	OFF	ON	ON	ON	ON	
15	H	X	H	H	H	H	H	OFF	OFF	OFF	OFF	OFF	OFF	OFF	
BI	X	X	X	X	X	X	L	OFF	OFF	OFF	OFF	OFF	OFF	OFF	2
RBI	H	L	L	L	L	L	L	OFF	OFF	OFF	OFF	OFF	OFF	OFF	3
LT	L	X	X	X	X	X	H	ON	ON	ON	ON	ON	ON	ON	4

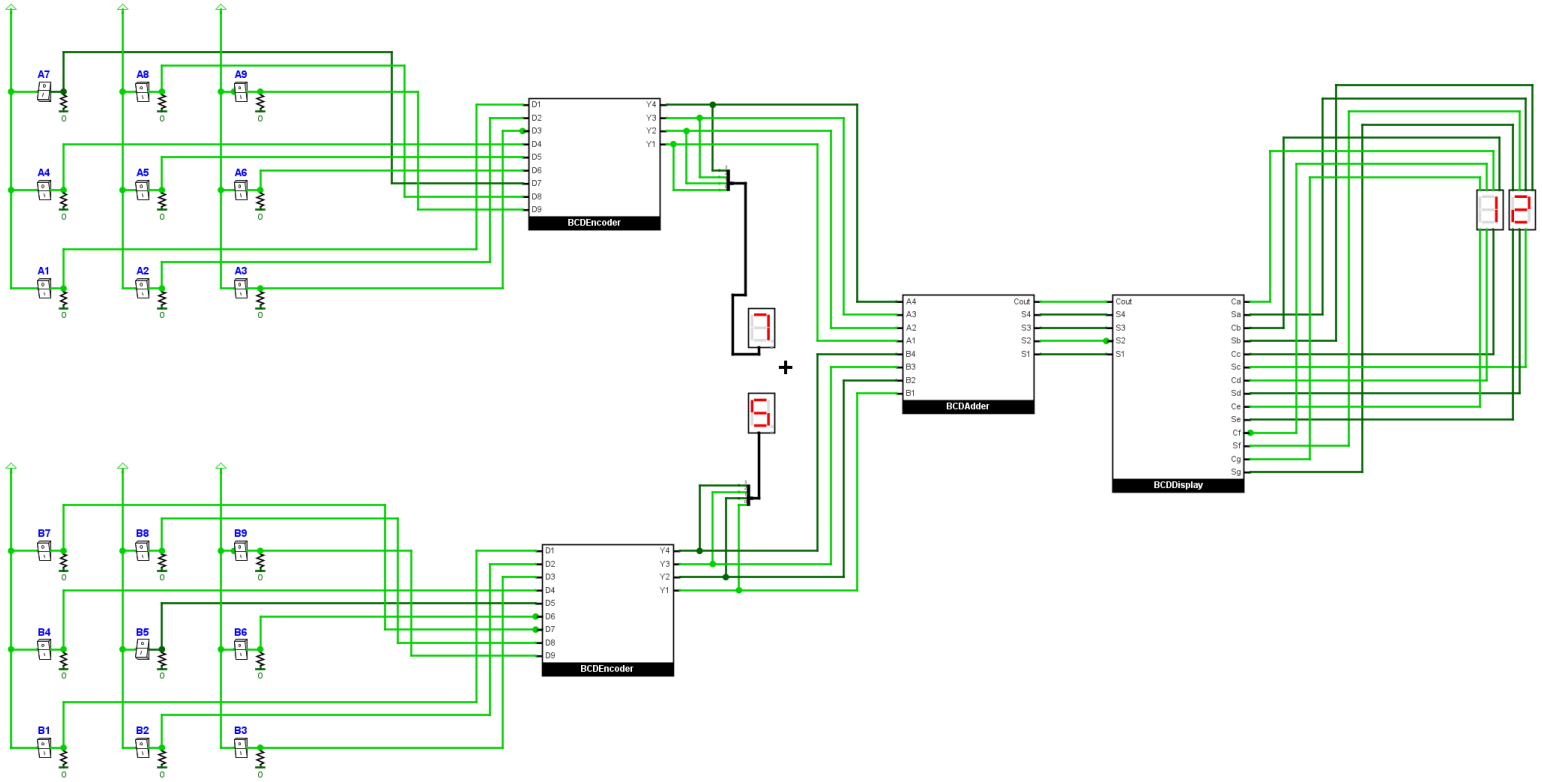
1.3 BCDDisplay Module | Active Low 7SEG







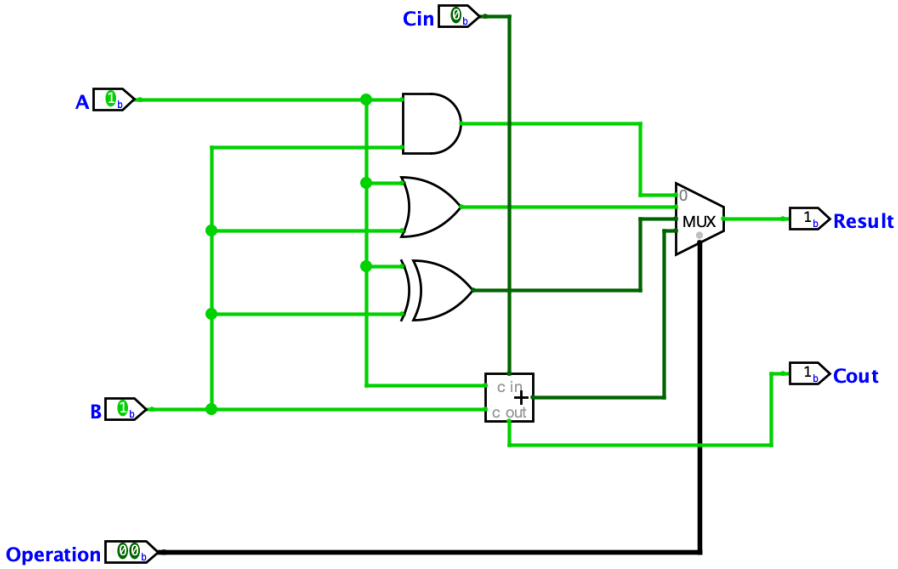
Checkpoint 1 (15 pts)



2. Basic ALU

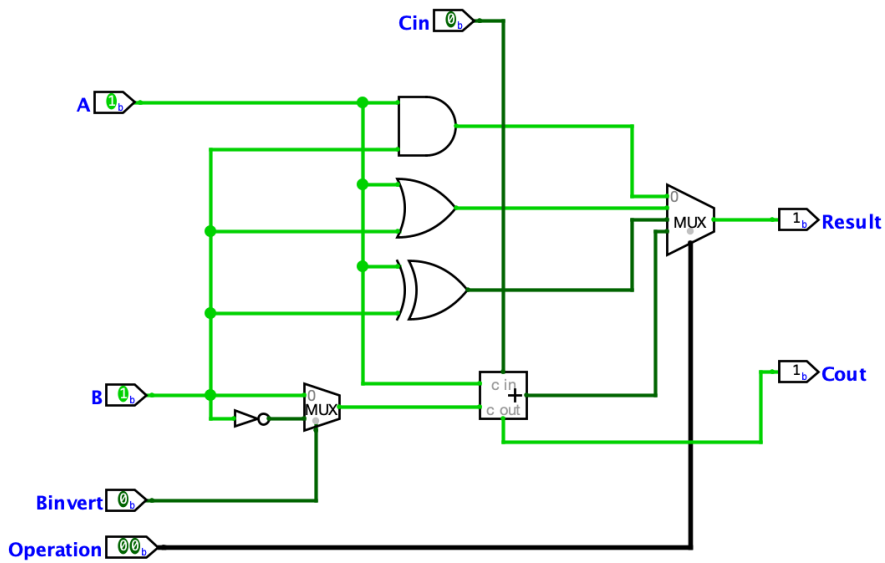
Arithmetic Logic Unit

2.1 1-Bit ALU



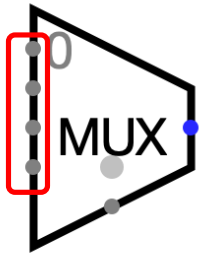
Operation	Result
00	AND
01	OR
10	XOR
11	Addition

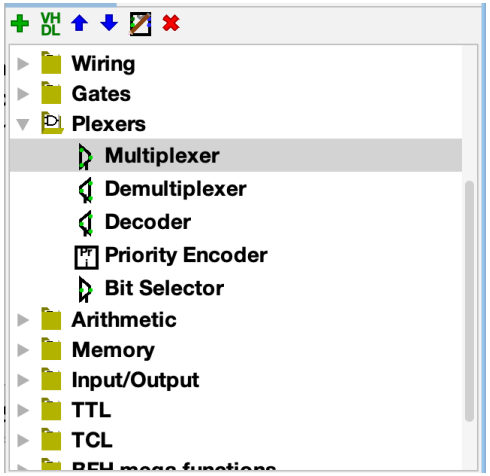
2.1 1-Bit ALU

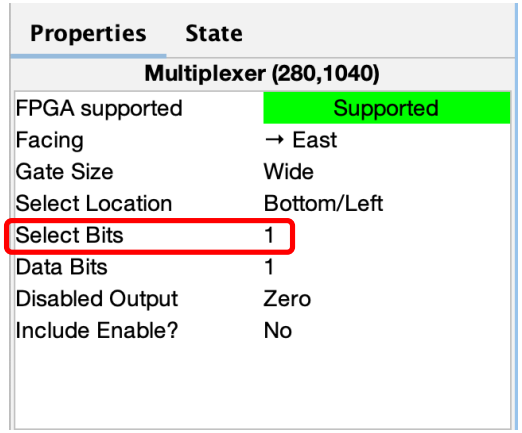


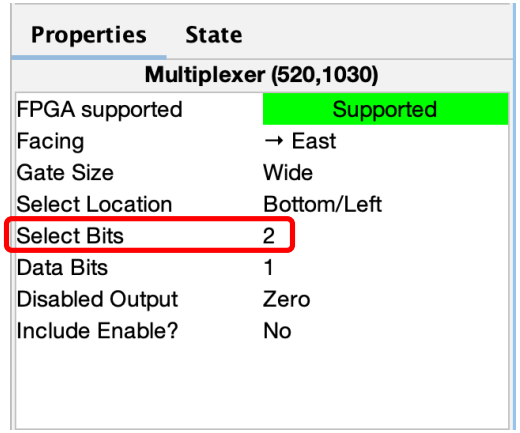
Binvert	Operation	Result
0	00	AND
0	01	OR
0	10	XOR
0	11	Addition
1	11	Subtraction

2.1 1 Bit ALU | MUX

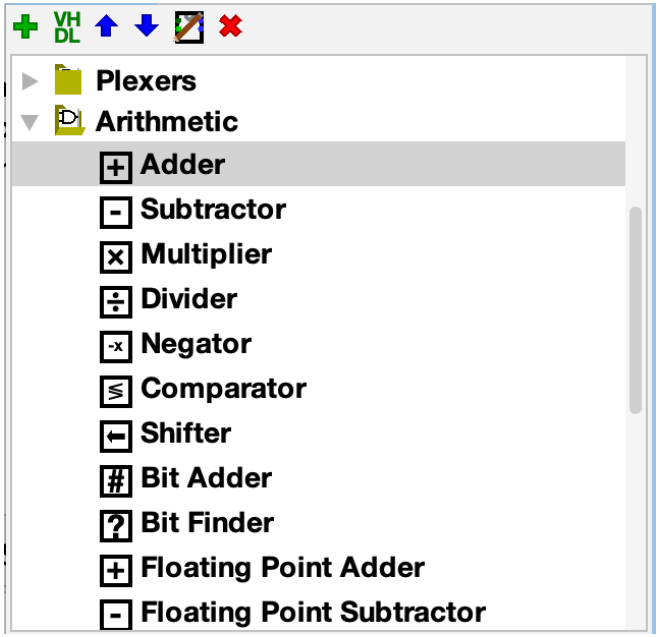


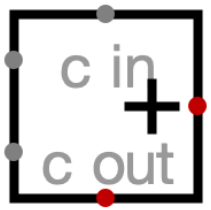


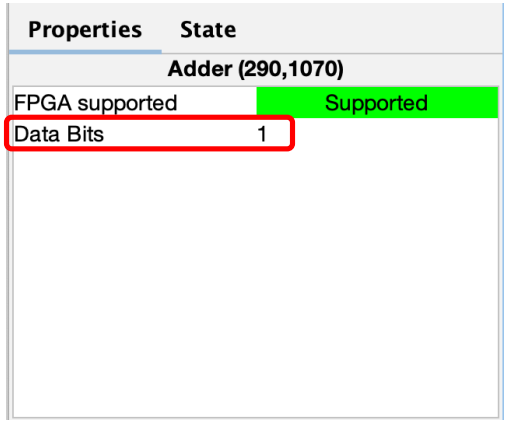




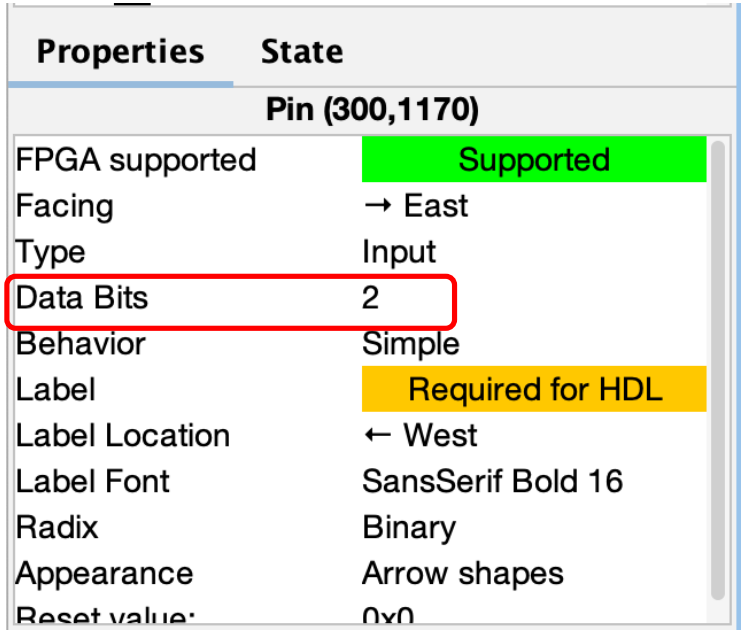
2.1 1-Bit ALU | Adder

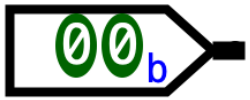




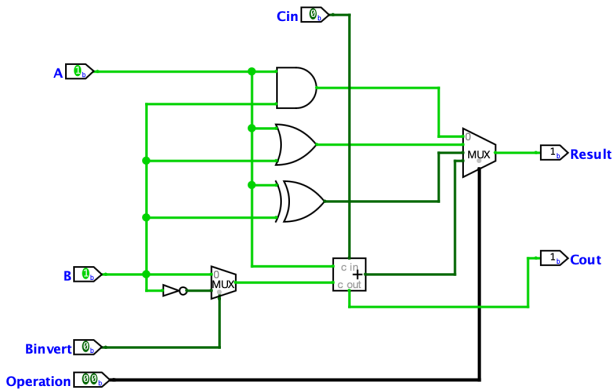


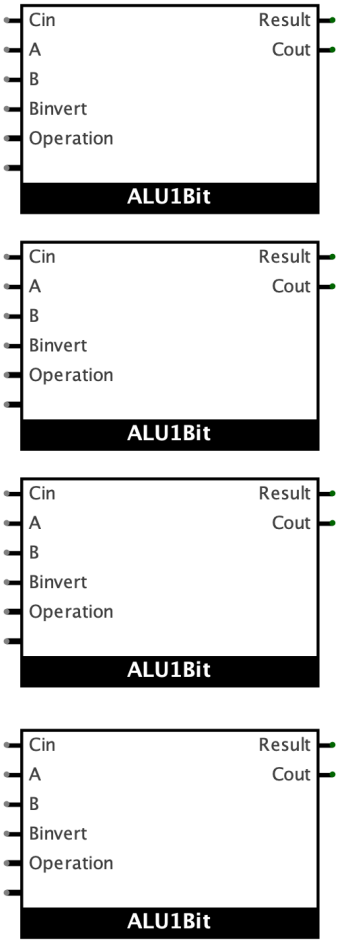
2.1 1-Bit ALU | Input Port



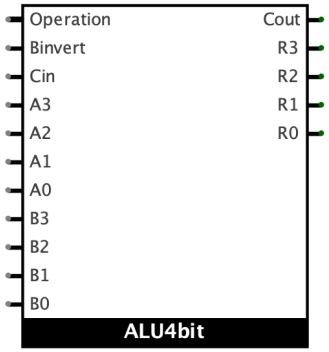


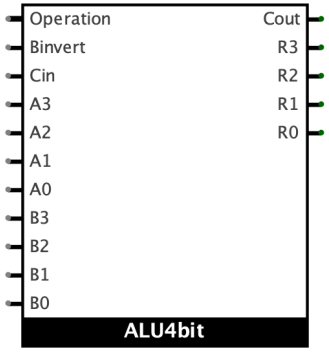
2.2 4-Bit ALU


Cin
A
B
Binvert
Operation
MUX
MUX
Result
Cout


Cin
A
B
Binvert
Operation
Result
Cout
ALU1Bit
Cin
A
B
Binvert
Operation
Result
Cout
ALU1Bit
Cin
A
B
Binvert
Operation
Result
Cout
ALU1Bit
Cin
A
B
Binvert
Operation
Result
Cout
ALU1Bit

2.3 8-Bit ALU | Checkpoint 2 (10 pts)





Lab 5 Submission

Date _____ Group No. _____

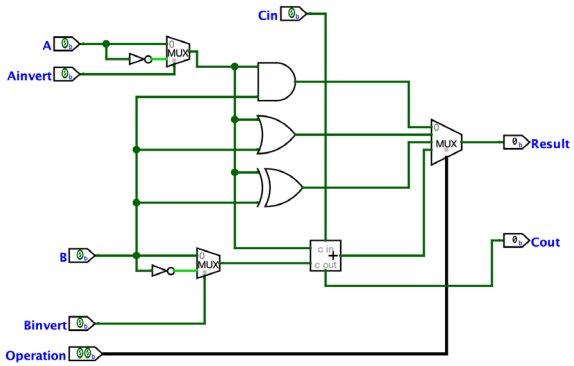
1. Student ID _____ Name _____

2. Student ID _____ Name _____

3. Student ID _____ Name _____

Questions

1. If **`Ainvert`** control signal has been added to 1-bit ALU as shown in the following figure, determine the results according to provided control signals and prove using Boolean algebra or truth table.



`Ainvert`	**`Binvert`**	Operation	Result	Prove
1	1	00		
1	1	01		

THANK YOU